

Fleet Risk Scoring

Sample task overview

Long-horizon Harbor coding environment · CPU ML engineering

This document covers what the task is, what the agent has to do, how it is graded, how frontier models performed and what ships in the folder. The task files and full rollout traces are included alongside it.

Task identification

Field	Overview
Task ID	fleet-risk-scoring-mini
Category	CPU ML engineering: feature engineering, outcome labelling under censoring and exact reproduction of a gradient-boosting risk service
Scenario	An airline maintenance desk's grounding risk scorer, broken by a migration that stripped five modules, has to be rebuilt from a formula-free specification and one published run
Task format	Editable Python package, instruction, specification, published backtest output, comparison tool, hidden verifier and reference solutions
Real data	Bundled operations export: 575,329 operated flight legs and 7,246 grounding events, January–March 2024, plus fleet register and station reference
Compute	2 vCPU, 7 GB RAM, no GPU
Agent budget	9,000 seconds; verifier 3,000 seconds with no network
Agent harness	mini-swe-agent, used for both models' rollouts
Deliverable	The five rebuilt modules, which must reproduce the scores of both scheduled runs flight by flight

Description

The agent inherits a broken production ML service. A maintenance-control risk scorer puts a probability on every operated flight leg that its airframe will be grounded within three days. A migration removed five modules from the package, so every function in them raises `NotImplementedError` and nothing runs.

To solve it the agent has to do the **full engineer's job** of reconstructing a system from its contract. `SPEC.md` describes all 38 published features in business terms and deliberately gives no formulas; where a definition could go more than one way, the published backtest decides. The agent has to rebuild the labelling rule, the censoring rule that decides which flights are fit on, the aircraft, station and recent-form history features and the smoothed reliability rates, and then close every gap against 24,817 published scores until the backtest matches exactly.

The core technical difficulty is that **matching the backtest is necessary but not sufficient**. The grader also runs a live configuration that scores the last week of March, six weeks after the fit window closes, and

nothing is published for it. Only there do airframes, stations and routes appear that the history has never seen, so the fallback rules in section 5 of the specification fire for the first time. A rebuild that fits the backtest but guesses at those rules fails, and only the specification can tell the agent what they are.

The incident the agent inherits

- **Five stripped modules.** `observation.py` (labelling and the fit subset), `tail_history.py`, `station_history.py`, `recent.py` and `reliability.py`: nine functions that produce the label, the fit set and 16 of the 38 features. The pipeline, model, feature assembly and configuration code are intact and must be left alone.
- **A specification with no formulas.** `SPEC.md` is written the way the operations desk describes its model. Interval boundaries, fallback levels, smoothing and the station grounding rate all have to be pinned down by testing candidates against the published run.
- **Two feeds that do not overlap.** Groundings come from a separate ops feed and a cancelled leg never appears in the flight log, so the two must be matched by tail number and time to label outcomes.
- **A censored export.** The export stops on the day it was pulled, so flights near the end of the fit window have outcomes the history cannot yet show. Section 3 says such flights are excluded from fitting but leaves the rule to the rebuild.
- **A live window nobody can check.** `configs/current.yaml` scores 25–31 March 2024 against a fit window ending 15 February; 581 of its 47,096 rows carry an airframe the fit window has never seen.

What the agent has to do

The work is a reconstruction loop: implement to the specification, diff the backtest and resolve each ambiguous definition until the diff is empty, then reason separately about the live window.

Phase	What happens	Files and tools
1. Read the contract	<code>SPEC.md</code> , the pipeline's call signatures, the declared feature order and both run configurations.	<code>SPEC.md</code> , <code>pipeline.py</code> , <code>schema.py</code>
2. Labelling and censoring	Label a flight at risk if its airframe is grounded within the configured window of departure; keep only fit-window flights whose outcome the readable history already settles.	<code>observation.py</code>
3. History features	Aircraft record, station record at both ends and recent form, learned from the fit window only, with fleet-level fallbacks, zero counts and -1 where no grounding exists.	<code>tail_history.py</code> , <code>station_history.py</code> , <code>recent.py</code>
4. Reliability rates	Operator, route and departure-station at-risk rates pulled toward the fleet rate by the configured smoothing.	<code>reliability.py</code>
5. Reconcile the backtest	Run the accepted backtest and diff it against the 24,817 published scores; resolve every definition that moves the scores.	<code>run_scoring.py</code> , <code>tools/compare_scores.py</code> , <code>reference/</code>
6. Reason about the live run	Check that unseen airframes, stations and routes follow section 5, then run both scheduled configurations.	<code>configs/current.yaml</code>

The only feedback is the backtest diff. Every stripped quantity feeds a fitted gradient-boosting model, so one wrong definition moves nearly every score and is hard to localise. The live-window fallbacks cannot be seen from the backtest at all, since every aircraft it scores is one the fit window already knows.

Example prompt

Opening of the task instruction given to the agent:

Maintenance control wants the grounding risk scorer back. A migration stripped five modules out of src/fleetrisk/ and never restored them, so every function in those five raises NotImplementedError and nothing runs. Rebuild them. The service has to reproduce its last accepted run to the last decimal, on both of the scheduled configurations ... configs/current.yaml runs the live scoring period off the same history. It scores flying from well after the end of that history, and nothing is published for it, so nothing will tell you whether you have that one right.

The full instruction (task/instruction.md) also points to SPEC.md, the published output and the comparison tool, and puts the other modules, configs and export out of scope.

Time, QA and difficulty calibration

Difficulty was measured, not asserted: each model was run four times (pass@4) on the mini-swe-agent harness inside the real task environment, and every attempt was graded by the same verifier.

Model	Strict passes	Mean reward	Note
Claude Fable 5 (xhigh reasoning)	4/4	1.00	28–38 steps per run, 11–18 minutes
Grok 4.7 (xhigh reasoning)	1/4	0.25	19–133 steps per run, 3–58 minutes

Target profile: a clear frontier lead. The task is calibrated so that Grok 4.7 lands in a 25–50% band (1–2 of 4 attempts), solvable but discriminating, while the frontier Claude model solves it reliably. Here Claude Fable 5 solved 4 of 4 against Grok 4.7's 1 of 4, a 75-percentage-point lead in strict pass rate.

Pass bars. The verifier runs the candidate service itself on both scheduled configurations and compares every flight's risk score to hidden expected values. To pass, both windows must clear the bar:

Check	Bar
Per-flight tolerance	$ \text{produced} - \text{expected} \leq 0.001$
Backtest agreement (24,817 flights, 5–8 Feb 2024)	$\geq 99.9\%$ of rows within tolerance
Live agreement (47,096 flights, 25–31 Mar 2024)	$\geq 99.9\%$ of rows within tolerance
Reward	1 only if both windows pass; otherwise 0

Per-attempt results (values outside a bar in bold):

Attempt	Result	Backtest rows matched	Backtest agreement	Live rows matched	Live agreement	Steps
Fable 5 · 53EhVdt	Pass	24,817 / 24,817	100.0%	47,096 / 47,096	100.0%	28
Fable 5 · TyDdFVQ	Pass	24,817 / 24,817	100.0%	47,096 / 47,096	100.0%	31
Fable 5 · duEM6s3	Pass	24,817 / 24,817	100.0%	47,096 / 47,096	100.0%	29
Fable 5 · jcbDkg8	Pass	24,817 / 24,817	100.0%	47,096 / 47,096	100.0%	38
Grok 4.7 · ySykpzY	Pass	24,817 / 24,817	100.0%	47,096 / 47,096	100.0%	19
Grok 4.7 · 2B8CNDE	Fail	19,696 / 24,817	79.4%	7,993 / 47,096	17.0%	83
Grok 4.7 · cHAVSeV	Fail	18,992 / 24,817	76.5%	7,360 / 47,096	15.6%	123
Grok 4.7 · txGSZQv	Fail	20,264 / 24,817	81.7%	5,637 / 47,096	12.0%	133

Why the model failed

Grok 4.7's three failed attempts are the failure mode the task is designed to catch. In each one the agent **rebuilt all five modules and ran both configurations end to end**: the service imported cleanly, scored every flight in both windows, wrote a valid summary and produced scores that tracked the published file closely (about 0.98 correlation on the backtest in the agents' own diagnostics). None of that is reproduction. Each agent submitted while tools/compare_scores.py still showed only 165, 196 and 431 of 24,817 backtest rows matching, with a largest difference of 0.22–0.25 in probability.

The misses sit in the definitions the specification leaves to the published run. Two attempts (cHAVSeV and txGSZQv) computed the station grounding rate as the at-risk share of departures from that airport, whereas the specification describes how often aircraft based there were grounded, which the reference counts through each airframe's home station in the fleet register. The third (2B8CNDE) matched that definition but, for example, gave a zero recent grounding rate to aircraft that flew in the fit window but not in its final fortnight, where the reference uses the fleet rate. Because each of these features feeds the fitted model, the error spread across the whole backtest and, further from the fit window, to 83–88% of the live rows. One agent closed by attributing the gap to "definitions the signed-off modules and spec do not leave free".

Every Fable 5 run started from a similar first diff (166–283 matching rows) and kept testing candidate definitions against the published run until the diff was empty; in run 53EhVdt the last change was opening the labelling window at departure, after which the backtest file was byte-identical. The pass rate reflects whether an agent treats "close" as done or keeps reasoning until a contract is met exactly.

Grading design

Reward is binary (1 only if both windows pass), but the verifier reports each window separately, with row counts and rank-quality diagnostics, so every attempt yields a dense record rather than a single bit.

Component	Overview
Execution	The grader copies the workspace, replaces both run configurations with verifier-held pinned copies and runs run_scoring.py itself (up to 2,400 seconds per run). Output files the agent left behind are never read.
Technical result	Every flight's risk_score is compared with hidden expected values at a tolerance of 0.001; missing flights count as misses and a repeated flight_id scores zero.

Component	Overview
Two windows	The accepted backtest and the live run must each reach 99.9% agreement. The backtest scores ship with the workspace; the live window, six weeks past the fit window, is the only place the section 5 fallbacks are exercised.
Diagnostics	Average precision and ROC AUC against the settled labels are reported for each window for diagnosis and do not gate the reward.
Isolation	Expected values and pinned configurations live only in tests/. The image build refuses to complete if any verifier material or a completed out/ directory is present in the workspace.

Failure, retries and limits

How we determine the agent has failed

Grading is deterministic and strictly binary: reward is 1.0 (pass) or 0.0 (fail). There is no submit step. When the attempt ends, the verifier copies the workspace, replaces both run configurations with its own pinned copies and runs the service itself, so no output file the agent left behind is read.

Reward is 1.0 only if both runs reproduce: at least 99.9% of the 24,817 backtest flights **and** at least 99.9% of the 47,096 live flights within 0.001 of the hidden expected scores. Miss either and the reward is 0.0, whether the service does not run, a definition moves the backtest, or the fallbacks break only on the unpublished live window.

Retries

- **No free retries of the outcome.** Within an attempt the agent can iterate as often as it likes: run the backtest, diff it against the published scores with the comparison tool and revise. That feedback covers the backtest only; nothing is returned for the live window.
- **One grading.** Only the final state of the workspace is graded, once, after the attempt ends.
- **Across attempts:** pass@4 means four fully independent attempts, each in a fresh container. They are independent samples used to measure the solve rate, not retries the agent controls. The target is Grok 4.7 in the 25–50% band (1–2 of 4) with the frontier Claude model well ahead; this task measured 4 of 4 for Claude Fable 5 and 1 of 4 for Grok 4.7.

Step and time limits

- **Time limit:** a hard 9,000 seconds (2 h 30 min) per attempt. An attempt still running at the limit is stopped.
- **Step limit:** none. The mini-swe-agent harness enforces no per-attempt step cap, so an attempt runs until the agent finishes or reaches the time limit.
- **Running out of time is a fail, not an error.** An attempt that ends without a service that reproduces both runs, including one stopped at the time limit, is graded 0. Genuine infrastructure failures, such as a container or harness fault, are reported separately as inconclusive and are never recorded as 0.

QA process

The task passes the same automated QA pipeline as the rest of the pack. Evidence for this task ships in the rollout folders:

- **Build and gates.** The Docker image builds reproducibly for each model's evaluation run (digests sha256:644981b9... at 496 MB and sha256:77924e4c... at 488 MB, recorded in the gates evidence). In both, the reference solution scores 1 and the untouched starting state scores 0.
- **Three structurally different positive solutions.** The reference solution labels by a per-aircraft sorted search; the first alternate uses a forward merge_asof against the grounding feed, blends rates as sums over counts and accumulates the aircraft record with bincount; the second resolves labels over contiguous runs of a sorted key, builds rate tables through an explicit code map and takes recent form from a reverse cumulative count. Three perturbed copies of the reference (renamed locals, reordered statements, added whitespace) check that incidental changes still reproduce the published run.
- **Ten negative controls, each capped at 0.05.** Every control except the first builds a complete working service and then introduces a single fault (backtest / live agreement as measured): doing nothing (the service does not start); unseen airframes dropped to zero instead of the fleet level (1.000000 / 0.986687, a perfect backtest and a failed live run); fitting on censored flights as if they were safe (0.699077 / 0.029578); the same fault hidden by repointing the live config at the backtest window (0.699077 / 0.029578, because the grader pins its own configs); the labelling window open at its far end (0.754443 / 0.162795); a labelling window one day too long (0.744933 / 0.100858); recent form over the wrong number of legs (0.732240 / 0.117483); unsmoothed rates (0.745497 / 0.118991); an entry point that republishes the shipped backtest scores (1.000000 / 0.000000); and a constant fleet base rate for every flight (0.000484 / 0.004501).

Environment and execution

Python 3.12 slim container (pandas 2.3.3, NumPy 2.2.6, scikit-learn 1.7.2, PyYAML 6.0.3) running as an unprivileged user. The agent works in /workspace/target/: run_scoring.py, SPEC.md and README.md, two run configurations, the export under data/, the fleetrisk package (20 files, five of them stubs), the published backtest under reference/ and tools/compare_scores.py.

The intact pipeline fixes the runtime contract: a HistGradientBoostingClassifier (seed 5, 200 iterations, learning rate 0.08, depth 6) on the 38 declared features, writing one row per flight in flight_id order with risk_score rounded to six decimals and a watch flag at 0.08.

Raw base data

Field	Overview
Source	US Department of Transportation, BTS On-Time Performance, January–March 2024 (public domain); four carriers (AA, DL, OO, WN)
Flight log	flights.csv.gz, 11.4 MB compressed: 575,329 operated legs from 1 January to 31 March 2024 across four operators (AA, DL, OO, WN) and 1,906 airframes
Grounding feed	groundings.csv.gz: 7,246 cancellation events with tail number, time and reason code (A, B, C) covering 1,327 airframes
Reference tables	fleet.csv: 1,906 airframes with operator and home station; stations.csv: 275 airports with city and state
Fit window	1 January – 15 February 2024; 262,881 legs are fittable once unsettled outcomes are censored

Field	Overview
Backtest window	5–8 February 2024: 24,817 scored flights, published with its run summary in reference/
Live window	25–31 March 2024: 47,096 scored flights, 581 of them on airframes the fit window has never seen; nothing published

Technical domains / relevant benchmarks

The task combines MLE-bench-style ML engineering (outcome labelling, censoring, smoothed target encoding, point-in-time history features) with SWE-bench and DeepSWE-style multi-file implementation against an existing codebase and Terminal-Bench-style autonomous terminal execution. Reconstructing a system exactly from a prose specification and one published result also relates it to PaperBench-style replication, applied to a production service where part of the graded behaviour cannot be observed before submission.

Folder contents

Folder	Contents
task/	The complete Harbor task: instruction.md, task.toml, environment (Dockerfile and workspace), tests (verifier, pinned configurations and hidden expected scores for both windows), solution (reference, two alternate solutions and three perturbed copies) and _private (negative controls with their expected caps).
Fable 5 pass@4/	agg/ holds the aggregate report and every trial's trajectory and verifier output; gold/ holds the Docker build log, image digest and the oracle and no-op gate results.
Grok 4.7 pass@4/	agg/ holds the aggregate report and every trial's trajectory and verifier output; gold/ holds the Docker build log, image digest and the oracle and no-op gate results.

How the task was created

The complete scoring service was written first and its accepted backtest published from it. The five modules carrying the label, the censoring rule and the history and reliability features were then removed, and the specification was written in the desk's business language with no formulas. The live window was placed six weeks past the fit window so the unseen-entity fallbacks fire only where nothing is published. It was then packaged with three independent correct implementations, perturbed copies, the negative-control battery and verifier-pinned configurations.

Acknowledgement

I confirm that I have reviewed the tasks and believe they are realistic, technically substantive, clearly specified, and appropriately graded. They reflect genuine professional workflows and are not low-effort, artificial, or "slop" tasks.

Name: Suleiman Mahmood

Role: Co-Founder

Signature: /s/Suleiman Mahmood

Date: 9.28.26
